

系统开发工具基础第一周实验报告

姓名：刘静雅

学号：24160001055

日期：2026 年 8 月 31 日

仓库：<https://github.com/Llj159-lab/sys-dev-tools-lab>

目录

1	实验概述	2
2	课上实验一：含空格文件名的批量整理	2
3	课上实验二：随机访问日志统计	3
4	课上实验三：制造、解决并解释一次 Git 合并冲突	5
5	课上实验四：修复并构建一页技术说明	6
6	课后练习	8
6.1	练习 1	8
6.2	练习 2	9
6.3	练习 3	10
6.4	练习 4	11
6.5	练习 5	11
6.6	练习 6	12
6.7	练习 7	12
6.8	练习 8	13
6.9	练习 9	14
6.10	练习 10	14
6.11	练习 11	15
6.12	练习 13	15
6.13	练习 14	16
6.14	练习 16	16
6.15	练习 17	17
6.16	练习 18	17
7	Git 提交记录	18
8	实验总结与感悟	18

1 实验概述

本次第一周实验围绕类 Unix Shell、Git 版本控制和 LaTeX 文档编辑展开，内容覆盖了文件系统操作、文本处理、分支合并、冲突解决和规范化排版等基础能力。四道课上实验强调“可复现”和“可验证”，课后练习则进一步巩固了 Shell 的基本语法、重定向、条件判断、管道、xargs、curl、jq、awk 等常用工具。

2 课上实验一：含空格文件名的批量整理

题目

在当前目录新建 q01，并完全使用命令行完成下列任务：创建 input/docs 和 input/tmp；创建 input/docs/notes one.txt（内容为 alpha 和 beta 两行）、input/docs/.secret.txt（内容为 hidden）、input/tmp/empty.txt（空文件）以及 input/run.log（任意两行日志）；显示 q01 的绝对路径，并以长格式列出 input 下全部项目，包含隐藏文件；把 input 下所有 .txt 文件复制到 work/< 学号 >/，保留相对目录结构并正确处理名称中的空格；将复制后的目录权限设为 750、普通文件权限设为 640；生成 inventory.txt，列出相对路径和字节数。

解析

这一题的重点不在命令复杂，而在细节是否准确。含空格文件名必须加引号，隐藏文件必须以点号开头，复制后还要保留目录结构，并检查权限是否按题目要求设置。整个过程的核心是用命令行完成创建、整理、验证三个步骤，而不是依赖手工处理。

```
mkdir -p input/docs input/tmp
printf 'alpha\nbeta\n' > "input/docs/notes one.txt"
printf 'hidden\n' > input/docs/.secret.txt
touch input/tmp/empty.txt
echo -e "log line 1\nlog line 2" > input/run.log

pwd
ls -la input/

find input -type f -name '*.txt' \
    -exec cp --parents -- '{}' work/24160001055/ \;

find work/24160001055 -type d -exec chmod 750 {} \;
find work/24160001055 -type f -exec chmod 640 {} \;
find work/24160001055 -type f -printf '%P %s\n' | sort > inventory.txt
```

结果

该题最终应能看到 `notes one.txt`、`.secret.txt`、`empty.txt` 和 `run.log` 等文件，其中复制目录只保留 `.txt` 文件，且目录与文件权限分别为 750 和 640。

```
luoyue@LAPTOP-20PGDGAI:~$ cd ~/sys-dev-tools-lab/week01/q01
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/q01$ pwd
/home/luoyue/sys-dev-tools-lab/week01/q01
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/q01$ ls -la input
total 20
drwxr-xr-x 4 luoyue luoyue 4096 Aug 24 14:47 .
drwxr-xr-x 5 luoyue luoyue 4096 Aug 30 21:19 ..
drwxr-xr-x 2 luoyue luoyue 4096 Aug 30 23:25 docs
-rw-r--r-- 1 luoyue luoyue 22 Aug 30 21:16 run.log
drwxr-xr-x 2 luoyue luoyue 4096 Aug 24 14:47 tmp
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/q01$ ls -la input/docs
total 16
drwxr-xr-x 2 luoyue luoyue 4096 Aug 30 23:25 .
drwxr-xr-x 4 luoyue luoyue 4096 Aug 24 14:47 ..
-rw-r--r-- 1 luoyue luoyue 7 Aug 30 23:25 .secret.txt
-rw-r--r-- 1 luoyue luoyue 11 Aug 30 23:24 'notes one.txt'
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/q01$ ls -la input/tmp
total 8
drwxr-xr-x 2 luoyue luoyue 4096 Aug 24 14:47 .
drwxr-xr-x 4 luoyue luoyue 4096 Aug 24 14:47 ..
-rw-r--r-- 1 luoyue luoyue 0 Aug 30 21:16 empty.txt
```

图 1: q01 文件结构检查

```
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/q01$ find work/24160001055 -exec stat -c
'%a %n' {} \; | sort
640 work/24160001055/input/docs/.secret.txt
640 work/24160001055/input/docs/notes one.txt
640 work/24160001055/input/tmp/empty.txt
750 work/24160001055
750 work/24160001055/input
750 work/24160001055/input/docs
750 work/24160001055/input/tmp
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/q01$ cat inventory.txt
input/docs/.secret.txt 7
input/docs/notes one.txt 11
input/tmp/empty.txt 0
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/q01$ if diff -u \
> <(find work/24160001055 -type f -printf '%P %s\n' | sort) \
> <(sort inventory.txt); then
> echo "inventory check: PASS"
> else
> echo "inventory check: FAIL"
> fi
inventory check: PASS
```

图 2: q01 权限与 inventory 检查

3 课上实验二：随机访问日志统计

题目

在 q02 中创建 `access.csv`，并使用下方给定数据完成日志统计。给定内容为：

```
timestamp,user,path,status,latency_ms
09:00:01,alice,/api/users,200,120
09:00:02,bob,/api/orders,500,310
09:00:03,alice,/api/users,503,180
09:00:04,carol,/login,500,90
09:00:05,bob,/api/orders,502,260
09:00:06,dave,/health,200,20
09:00:07,alice,/api/orders,500,420
09:00:08,carol,/api/users,500,150
```

题目要求：编写 `analyze.sh`，接收一个 CSV 文件路径；文件不存在时将错误写入 `stderr` 并返回非零退出码；使用 Shell 管道以及 `awk`、`sort`、`head` 等工具，输出 HTTP 5xx 数量最多的前 2 个 path，按次数降序，次数相同时按 path 字典序；输出全部数据行的平均 `latency_ms`，保留两位小数；分别对 `access.csv` 和一个不存在的文件运行脚本；不得使用 Python、电子表格或手工统计。

解析

这一题要求把文本数据拆成多个小步骤处理。先用 `awk` 过滤 5xx 行并提取 path，再用 `sort` 和 `uniq -c` 做计数，最后再按题目要求排序。平均延迟则需要排除表头，只对数据行做求和与计数。脚本还必须考虑异常输入，这是命令行工具是否规范的重要部分。

```
awk -F, 'NR>1 && $4 ~ /^5/ {print $3}' "$1" |
sort |
uniq -c |
LC_ALL=C sort -k1,1nr -k2,2 |
head -2

awk -F, 'NR>1 {sum += $5; count++}
END {printf "%.2f\n", sum/count}' "$1"
```

结果

根据给定数据，结果应为 `/api/orders` 和 `/api/users`，平均延迟为 193.75 ms。

```

inventory check: PASS
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/q01$ cd ~/sys-dev-tools-lab/week01/q02
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/q02$ ./analyze.sh access.csv
=== HTTP 5xx 数量最多的前2个path ===
    3 /api/orders
    2 /api/users

=== 全部数据行的平均延迟 (ms) ===
193.75
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/q02$

```

图 3: q02 正常输入运行结果

```

luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/q02$ ./analyze.sh nonexistent.csv
Error: 文件 'nonexistent.csv' 不存在
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/q02$ rc=$?
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/q02$ printf 'exit_code=%s\n' "$rc"
exit_code=1

```

图 4: q02 不存在文件测试

4 课上实验三：制造、解决并解释一次 Git 合并冲突

题目

在 q03 中从零创建一个 Git 仓库，并主动制造、解决一次合并冲突。题目要求：初始化仓库，在 main 创建 config.txt，内容为 mode=normal，并完成初始提交；创建 feature-a，把内容改为 mode=safe 并提交；从初始 main 创建 feature-b，把内容改为 mode=fast 并提交；回到 main，先合并 feature-a，再合并 feature-b；解决冲突时保留 mode=safe，并另加一行 note=reviewed；确认工作区干净，运行 git log --all --graph --decorate --oneline 查看历史。

解析

这一题的重点是理解分支从哪里创建、合并是什么顺序发生，以及冲突为什么出现。两个分支修改同一位置时，Git 不能自动判断最终保留哪一版，就会产生冲突。解决冲突并不是简单消除报错，而是要明确最终应保留的内容及其原因。

```

git init -b main
printf 'mode=normal\n' > config.txt
git add config.txt
git commit -m "初始提交: mode=normal"

git checkout -b feature-a
printf 'mode=safe\n' > config.txt
git commit -am "feature-a: 修改为 safe 模式"

```

```
git checkout main
git checkout -b feature-b
printf 'mode=fast\n' > config.txt
git commit -am "feature-b: 修改为 fast 模式"
```

结果

最终应保留 mode=safe 和 note=reviewed，并且提交历史应清晰展示分支与合并关系。

```
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/q03$ cd ~/sys-dev-tools-lab/week01/q03
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/q03$ git branch --show-current
main
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/q03$ git branch --list
feature-a
feature-b
* main
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/q03$ git log --all --graph --decorate --oneline
* 3dffa07 (HEAD -> main) 合并 feature-b: 解决冲突并保留 safe 模式
|
| * 5518895 (feature-b) feature-b: 修改为 fast 模式
| * | 1966d0f 合并 feature-a
|/
|/
| * d917153 (feature-a) feature-a: 修改为 safe 模式
|/
* f817556 初始提交: mode=normal
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/q03$
```

图 5: q03 Git 分支历史

```
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/q03$ cat config.txt
mode=safe
note=reviewed
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/q03$ if [ -z "$(git status --porcelain)"
]; then
> echo "working tree: CLEAN"
> else
> git status
> fi
working tree: CLEAN
```

图 6: q03 最终文件内容和工作区状态

5 课上实验四：修复并构建一页技术说明

题目

将下方模板保存为 q04/report.tex，补全内容并从命令行构建 PDF。

```
\documentclass{article}

\usepackage{amsmath}
```

```

\begin{document}

\title{Tool Report}

\author{姓名—学号}

\maketitle

\section{Result}

% 在此加入公式、表格及正文引用

\end{document}

```

题目要求：加入一个带编号和 label 的公式，并在正文中使用 ref 或 eqref 引用；加入一个 2 行 2 列表格，设置 caption 和 label，并在正文中引用该表；使用 latexmk -pdf -halt-on-error 或 tectonic 生成 report.pdf；确认 PDF 能打开，且交叉引用不显示 ??。

解析

这一题的核心是掌握 LaTeX 的结构化排版方式。公式和表格不是手工编号，而是通过 label 与 ref 自动维护。编译时需要注意交叉引用往往要经过多轮才能正确生成，因此用 latexmk 比单次编译更稳妥。

公式~\ref{eq:einstein} 展示了质能方程。

```

\begin{equation}
E = mc^2
\label{eq:einstein}
\end{equation}

```

表格~\ref{tab:example} 展示了一个简单的2行2列数据。

结果

最终 PDF 中应能够正常显示公式编号、表格编号和交叉引用，不应出现 ??。

```

luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/q04$ if grep -nE 'undefined|Undefined|The
re were undefined references|\?\?' \
> report.log report.aux report.out; then
> echo "reference check: FAIL"
> else
> echo "reference check: PASS"
> fi
reference check: PASS
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/q04$ ls -lh report.pdf

```

图 7: q04 LaTeX 编译与引用检查

Tool Report

刘静雅 - 24160001055

2026 年 8 月 31 日

1 Result

公式 1 展示了质能方程。

$$E = mc^2 \tag{1}$$

表格 1 展示了一个简单的 2 行 2 列数据：

表 1: 示例表格

项目	数值
A	10
B	20

图 8: q04 PDF 预览

6 课后练习

6.1 练习 1

题目：本课程要求你使用类 Unix 的 Shell，如 Bash 或 ZSH。若你在 Linux 或 macOS 上，无需额外设置。若你在 Windows 上，请确认你用的不是 `cmd.exe` 或 `PowerShell`；你可以使用 <https://docs.microsoft.com/en-us/windows/wsl/> 或 Linux 虚拟机来获得 Unix 风格的命令行工具。要确认当前 Shell 是否合适，可运行 `echo $SHELL`；若输出类似 `/bin/bash` 或 `/usr/bin/zsh`，就说明没问题。

解析：本题是在确认实验环境。Shell 是后续所有练习的基础，如果使用的是不兼容的命令行环境，很多语法、重定向和脚本行为都会不同，因此先确认当前 Shell 是否为 Bash 或 Zsh 是必要步骤。

```
luoyue@LAPTOP-20PGDGAI:~$ echo $SHELL
/bin/bash
luoyue@LAPTOP-20PGDGAI:~$ ps -p $$
  PID TTY          TIME CMD
 7829 pts/5        00:00:00 bash
luoyue@LAPTOP-20PGDGAI:~$
```

图 9: 练习 1: 确认当前 Shell 环境

6.2 练习 2

题目：ls 的 -l 选项 (flag) 作用是什么？运行 `ls -l /` 并观察输出。每一行最前面的 10 个字符分别代表什么？（提示：man ls）

解析：这一题主要是理解长格式输出。最前面的字符包含文件类型和权限信息，后续字段则与文件所有者、所属组、大小和时间有关。通过实际输出可以把权限位从抽象概念转化为可直接观察的结果。

```

luoyue@LAPTOP-20PGDGAI:~$ ls -l /
total 2772
drwxr-xr-x  3 root root    4096 Aug  4 09:12 Docker
lrwxrwxrwx  1 root root      7 Apr 22 2024 bin -> usr/bin
drwxr-xr-x  2 root root    4096 Feb 26 2024 bin.usr-is-merged
drwxr-xr-x  2 root root    4096 Apr 22 2024 boot
drwxr-xr-x 15 root root   3860 Aug 30 21:14 dev
drwxr-xr-x 102 root root    4096 Aug 31 00:50 etc
-rw-r--r--  1 root root     32 Nov 14 2025 flag
drwxr-xr-x  3 root root    4096 Nov 14 2025 home
-rwxrwxrwx  1 root root 2735264 Aug  7 2025 init
lrwxrwxrwx  1 root root      7 Apr 22 2024 lib -> usr/lib
drwxr-xr-x  2 root root    4096 Apr  8 2024 lib.usr-is-merged
lrwxrwxrwx  1 root root      9 Apr 22 2024 lib64 -> usr/lib64
drwx----- 2 root root   16384 Nov 14 2025 lost+found
drwxr-xr-x  2 root root    4096 Aug  6 2025 media
drwxr-xr-x  6 root root    4096 Nov 14 2025 mnt
drwxr-xr-x  2 root root    4096 Aug  6 2025 opt
dr-xr-xr-x 344 root root      0 Aug 30 21:14 proc
drwx----- 5 root root    4096 Aug  4 09:12 root
drwxr-xr-x 21 root root    620 Aug 30 21:27 run
lrwxrwxrwx  1 root root      8 Apr 22 2024/sbin -> usr/sbin
drwxr-xr-x  2 root root    4096 Mar 31 2024/sbin.usr-is-merged
drwxr-xr-x  2 root root    4096 Nov 14 2025 snap
drwxr-xr-x  2 root root    4096 Aug  6 2025 srv
dr-xr-xr-x 13 root root      0 Aug 30 21:14 sys
drwxrwxrwt  9 root root   20480 Aug 31 00:25 tmp
drwxr-xr-x 12 root root    4096 Aug  6 2025 usr
drwxr-xr-x 13 root root    4096 Nov 14 2025 var

```

图 10: 练习 2: 观察 ls -l 输出

6.3 练习 3

题目: 在命令 `find /Downloads -type f -name "*.zip" -mtime +30` 中, *.zip 是一个“glob”。什么是 glob? 新建一个测试目录并创建一些文件, 试试 `ls *.txt`、`ls file?.txt`、`ls {a,b,c}.txt` 等模式。参见 Bash 手册中的 https://www.gnu.org/software/bash/manual/html_node/Pattern-Matching.html。

解析: glob 是 Shell 层面的文件名模式匹配机制, 不是正则表达式。本题通过不同模式的对比, 理解 *、? 和花括号展开的区别, 有助于后续批量处理文件。

```

luoyue@LAPTOP-20PGDGAI:~$ cd ~/sys-dev-tools-lab/week01
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01$ mkdir -p practice/globtest
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01$ cd practice/globtest
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/practice/globtest$ touch a.txt b.txt c.txt
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/practice/globtest$ ls *.txt
a.txt b.txt c.txt file1.txt file10.txt file2.txt
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/practice/globtest$ ls file?.txt
file1.txt file2.txt
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/practice/globtest$ ls {a,b,c}.txt
a.txt b.txt c.txt
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/practice/globtest$

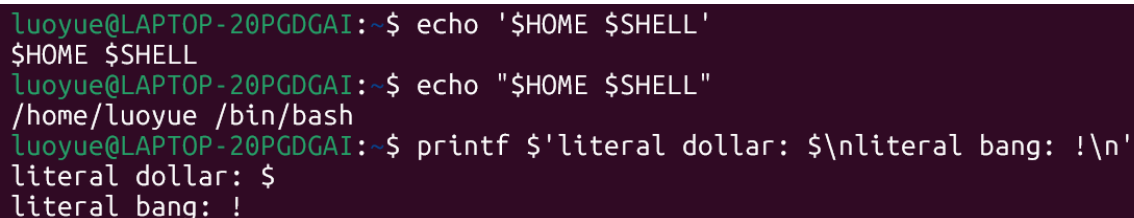
```

图 11: 练习 3: glob 文件名匹配

6.4 练习 4

题目： ' 单引号'、" 双引号" 和 \$'ANSI 引号' 有什么区别？写一条命令，输出一个同时包含字面量 \$、! 和换行符的字符串。参见 https://www.gnu.org/software/bash/manual/html_node/Quoting.html。

解析： 单引号保留字面量，双引号允许变量展开，ANSI 引号允许使用转义序列输出换行等特殊字符。理解这三种引用方式，是编写稳定 Shell 脚本的基础。

A terminal window with a dark purple background and light green text. It shows four commands and their outputs. The first command is 'echo '\$HOME \$SHELL'', which outputs '\$HOME \$SHELL'. The second command is 'echo "\$HOME \$SHELL"', which outputs '/home/luoyue /bin/bash'. The third command is 'printf \$'literal dollar: \$\\nliteral bang: !\\n'', which outputs 'literal dollar: \$' followed by a newline and 'literal bang: !' on the next line.

```
luoyue@LAPTOP-20PGDGAI:~$ echo '$HOME $SHELL'
$HOME $SHELL
luoyue@LAPTOP-20PGDGAI:~$ echo "$HOME $SHELL"
/home/luoyue /bin/bash
luoyue@LAPTOP-20PGDGAI:~$ printf $'literal dollar: $\\nliteral bang: !\\n'
literal dollar: $
literal bang: !
```

图 12: 练习 4: Shell 引号规则

6.5 练习 5

题目： Shell 有三条标准流：stdin (0)、stdout (1)、stderr (2)。运行 `ls /nonexistent /tmp`，把 stdout 和 stderr 分别重定向到两个文件。你将如何把两者都重定向到同一个文件？参见 https://www.gnu.org/software/bash/manual/html_node/Redirections.html。

解析： 这一题说明标准输出和标准错误是分开的。将它们分别重定向后，可以更清楚地区分正常信息和错误信息；而将二者合并到同一文件，则适合统一收集日志。

```

luoyue@LAPTOP-20PGDGAI:~$ cd ~/sys-dev-tools-lab/week01/practice
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/practice$ ls /nonexistent /tmp > stdout.txt 2> stderr.txt
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/practice$ echo "--- stdout.txt ---"
--- stdout.txt ---
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/practice$ cat stdout.txt
/tmp:
gv_report_pdf.0ZipfE
snap-private-tmp
systemd-private-6bb8ee13095b406ab6d8f0bc50b61cc4-polkit.service-0eujbs
systemd-private-6bb8ee13095b406ab6d8f0bc50b61cc4-systemd-logind.service-ebZLIi
systemd-private-6bb8ee13095b406ab6d8f0bc50b61cc4-systemd-resolved.service-53oBu0
systemd-private-6bb8ee13095b406ab6d8f0bc50b61cc4-systemd-timesyncd.service-J0RnWI
systemd-private-6bb8ee13095b406ab6d8f0bc50b61cc4-wsl-pro.service-e0VoLs
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/practice$ echo "--- stderr.txt ---"
--- stderr.txt ---
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/practice$ cat stderr.txt
ls: cannot access '/nonexistent': No such file or directory
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/practice$ ls /nonexistent /tmp > both.txt 2>&1
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/practice$ echo "--- both.txt ---"
--- both.txt ---
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/practice$ cat both.txt
ls: cannot access '/nonexistent': No such file or directory
/tmp:
gv_report_pdf.0ZipfE
snap-private-tmp
systemd-private-6bb8ee13095b406ab6d8f0bc50b61cc4-polkit.service-0eujbs
systemd-private-6bb8ee13095b406ab6d8f0bc50b61cc4-systemd-logind.service-ebZLIi
systemd-private-6bb8ee13095b406ab6d8f0bc50b61cc4-systemd-resolved.service-53oBu0
systemd-private-6bb8ee13095b406ab6d8f0bc50b61cc4-systemd-timesyncd.service-J0RnWI
systemd-private-6bb8ee13095b406ab6d8f0bc50b61cc4-wsl-pro.service-e0VoLs

```

图 13: 练习 5: 标准流重定向

6.6 练习 6

题目：\$? 保存上一条命令的退出状态（0 表示成功）。&& 仅在前一条成功时执行后一条；|| 仅在前一条失败时执行后一条。写一个一行命令：仅当 /tmp/mydir 不存在时才创建它。参见 https://www.gnu.org/software/bash/manual/html_node/Exit-Status.html。

解析：Shell 中的条件执行本质上是基于退出状态。本题展示了如何利用 || 实现“如果不存在就创建”的常见写法，这也是自动化脚本里非常常见的模式。

```

luoyue@LAPTOP-20PGDGAI:~$ test -d /tmp/mydir || mkdir /tmp/mydir
luoyue@LAPTOP-20PGDGAI:~$ echo "exit_code=$?"
exit_code=0
luoyue@LAPTOP-20PGDGAI:~$ ls -ld /tmp/mydir
drwxr-xr-x 2 luoyue luoyue 4096 Aug 31 00:57 /tmp/mydir
luoyue@LAPTOP-20PGDGAI:~$ test -d /tmp/mydir && echo "/tmp/mydir already exists"
/tmp/mydir already exists

```

图 14: 练习 6: 退出状态与条件执行

6.7 练习 7

题目：为什么 cd 必须是 Shell 内建命令，而不能是独立程序？（提示：想想子进程能影响和不能影响父进程的哪些状态。）

解析：该题强调进程关系。目录切换属于当前 Shell 的状态变化，如果由独立程序执行，改变的只会是子进程自己的工作目录，父 Shell 不会受到影响，因此 cd 必须由 Shell

自身实现。

```
luoyue@LAPTOP-20PGDGAI:~$ type cd
cd is a shell builtin
luoyue@LAPTOP-20PGDGAI:~$ pwd
/home/luoyue
luoyue@LAPTOP-20PGDGAI:~$ bash -c 'cd /tmp; pwd'
/tmp
luoyue@LAPTOP-20PGDGAI:~$ pwd
/home/luoyue
```

图 15: 练习 7: 验证 cd 是 Shell 内建命令

6.8 练习 8

题目: 写一个脚本, 接收文件名参数 (\$1), 用 `test -f` 或 `[ -f ... ]` 检查该文件是否存在, 并根据结果输出不同提示。参见 https://www.gnu.org/software/bash/manual/html_node/Bash-Conditional-Expressions.html。

解析: 这是最基础的 Shell 条件判断练习。通过参数 \$1 接收外部输入, 再用 `test -f` 判断文件是否存在, 可以把单条命令扩展成可复用脚本。

```
#!/usr/bin/env bash
if [ -f "$1" ]; then
    echo "file exists: $1"
else
    echo "file not found: $1"
fi
```

```
luoyue@LAPTOP-20PGDGAI:~$ cd ~/sys-dev-tools-lab/week01/practice
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/practice$ cat > check.sh <<'EOF'
> #!/usr/bin/env bash
> if [ -f "$1" ]; then
>     echo "file exists: $1"
> else
>     echo "file not found: $1"
> fi
EOF
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/practice$ touch somefile
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/practice$ bash check.sh somefile
file exists: somefile
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/practice$ bash check.sh missingfile
file not found: missingfile
```

图 16: 练习 8: 文件存在性检查脚本

6.9 练习 9

题目：把上一题完成的脚本保存为文件（如 `check.sh`）。先运行 `./check.sh somefile`，会发生什么？然后执行 `chmod +x check.sh` 再试一次。为什么这一步是必须的？（提示：比较 `chmod` 前后的 `ls -l check.sh` 输出）

解析：这一题说明文件内容正确并不等于可以执行。没有执行权限时，Shell 不能把脚本当作程序直接运行；赋予执行权限后，脚本才可以通过 `./` 调用。

```
luoyue@LAPTOP-20PGDGAI:~$ cd ~/sys-dev-tools-lab/week01/practice
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/practice$ ls -l check.sh
-rw-r--r-- 1 luoyue luoyue 106 Aug 31 00:59 check.sh
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/practice$ ./check.sh somefile
-bash: ./check.sh: Permission denied
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/practice$ chmod +x check.sh
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/practice$ ls -l check.sh
-rwxr-xr-x 1 luoyue luoyue 106 Aug 31 00:59 check.sh
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/practice$ ./check.sh somefile
file exists: somefile
```

图 17: 练习 9: `chmod +x` 前后的执行效果

6.10 练习 10

题目：在脚本的 `set` 选项（flag）里加入 `-x` 会发生什么？写个简单脚本试试并观察输出。参见 https://www.gnu.org/software/bash/manual/html_node/The-Set-Builtin.html。

解析：`set -x` 会在执行命令前打印展开后的命令，这对于调试脚本非常有用。它可以帮助定位变量替换、命令替换和分支执行过程中的问题。

```
#!/usr/bin/env bash
set -x
name="week01"
echo "current practice: $name"
date +%F
```

```

luoyue@LAPTOP-20PGDGAI: ~ $ cd ~/sys-dev-tools-lab/week01/practice
luoyue@LAPTOP-20PGDGAI: ~/sys-dev-tools-lab/week01/practice$ cat > debug.sh <<'EOF'
> #!/usr/bin/env bash
set -x

name="week01"
echo "current practice: $name"
date +%F
EOF
luoyue@LAPTOP-20PGDGAI: ~/sys-dev-tools-lab/week01/practice$ chmod +x debug.sh
luoyue@LAPTOP-20PGDGAI: ~/sys-dev-tools-lab/week01/practice$ ./debug.sh
+ name=week01
+ echo 'current practice: week01'
current practice: week01
+ date +%F
2026-08-31
luoyue@LAPTOP-20PGDGAI: ~/sys-dev-tools-lab/week01/practice$

```

图 18: 练习 10: set -x 调试脚本

6.11 练习 11

题目：写一条命令，把文件复制为带当天日期的备份文件名（例如 notes.txt → notes_2026-01-12.txt）。（提示：\$(date +%Y-%m-%d)）参见 https://www.gnu.org/software/bash/manual/html_node/Command-Substitution.html。

解析：命令替换可以把命令执行结果嵌入文件名中，这是一种常见的自动备份方式。它使备份文件名自动包含日期，便于归档和查找。

```
cp notes.txt "notes_$(date +%F).txt"
```

```

luoyue@LAPTOP-20PGDGAI: ~/sys-dev-tools-lab/week01/practice$ cd ~/sys-dev-tools-lab/week01/p
ractice
luoyue@LAPTOP-20PGDGAI: ~/sys-dev-tools-lab/week01/practice$ echo "backup demo" > notes.txt
luoyue@LAPTOP-20PGDGAI: ~/sys-dev-tools-lab/week01/practice$ cp notes.txt "notes_$(date +%Y-
%m-%d).txt"
luoyue@LAPTOP-20PGDGAI: ~/sys-dev-tools-lab/week01/practice$ ls -l notes*
-rw-r--r-- 1 luoyue luoyue 12 Aug 31 01:03 notes.txt
-rw-r--r-- 1 luoyue luoyue 12 Aug 31 01:03 notes_2026-08-31.txt
luoyue@LAPTOP-20PGDGAI: ~/sys-dev-tools-lab/week01/practice$

```

图 19: 练习 11: 日期备份

6.12 练习 13

题目：使用管道找出你“home 目录”中最常见的 5 种文件扩展名。（提示：组合 find、grep/sed/awk、sort、uniq -c 以及 head）

解析：这一题综合了文件查找、文本清洗、排序和计数。扩展名统计并不复杂，但能很好地体现管道把多个小工具组合成一个完整分析流程的能力。

```

find ~ -type f 2>/dev/null |
sed -n 's/.*\./p' |
sort |
uniq -c |

```

```
sort -nr |  
head -5
```

```
luoyue@LAPTOP-20PGDGAI:~$ find ~ -type f 2>/dev/null | sed -n 's/.*\./p' | sort | uniq -c  
| sort -nr | head -5  
26261 py  
8207 asm  
5400 pyc  
2404 json  
1060 txt
```

图 20: 练习 13: 文件扩展名统计

6.13 练习 14

题目: xargs 会把 stdin 的每一行转换为命令参数。结合 find 和 xargs (不要用 find -exec), 找出目录中所有 .sh 文件, 并用 wc -l 统计每个文件行数。加分项: 正确处理文件名中的空格。(提示: -print0 和 -0) 参见 <https://man7.org/linux/man-pages/man1/xargs.1.html>。

解析: 这题的重点是正确处理文件名中的空格。使用 -print0 与 -0 可以让 xargs 按空字符分隔参数, 避免普通空格把文件名切碎。

```
find . -type f -name '*.sh' -print0 |  
xargs -0 -I{} sh -c 'printf "%s: " "$1"; wc -l < "$1" sh "{}'
```

```
luoyue@LAPTOP-20PGDGAI:~$ find . -type f -name '*.sh' -print0 | xargs -0 -I{} sh -c 'printf  
"%s: " "$1"; wc -l < "$1" sh "{}'  
./local/share/uv/python/cpython-3.13.14-linux-x86_64-gnu/lib/itcl4.3.5/itclConfig.sh: 67  
./local/share/pipx/venvs/virtualenv/lib/python3.12/site-packages/virtualenv/activation/bas  
h/activate.sh: 114  
./q02/analyze.sh: 42  
./d20-run-v1/experiments/scripts/run_d10_tasks.sh: 46  
./d20-run-v1/experiments/scripts/run_d15_heldout_tasks.sh: 102  
./d20-run-v1/experiments/scripts/run_d11_tasks.sh: 47  
./d20-run-v1/experiments/scripts/run_d15_heldout_recovery_tasks.sh: 101  
./d20-run-v1/experiments/scripts/run_d20_final_three_arm.sh: 90  
./d20-run-v1/experiments/scripts/run_d20_valid_two_comparison.sh: 45  
./d20-run-v1/.venv-wsl/lib/python3.12/site-packages/tqdm/completion.sh: 19  
./d20-run-v1/.venv-wsl/lib/python3.12/site-packages/litellm/proxy/start.sh: 1  
./d20-run-v1/.venv/Lib/site-packages/tqdm/completion.sh: 19  
./d20-run-v1/.venv/Lib/site-packages/litellm/proxy/start.sh: 1  
./sys-dev-tools-lab/week01/practice/check.sh: 6  
./sys-dev-tools-lab/week01/practice/debug.sh: 6  
./sys-dev-tools-lab/week01/q02/analyze.sh: 14
```

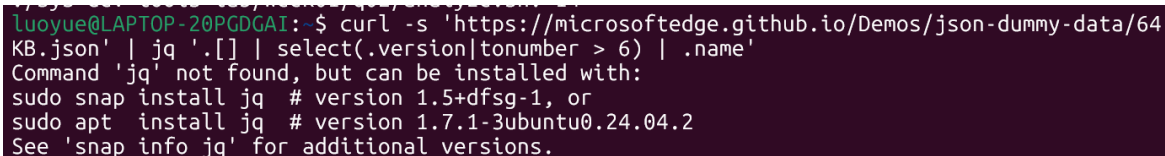
图 21: 练习 14: xargs 统计 Shell 脚本行数

6.14 练习 16

题目: jq 是处理 JSON 的强大工具。用 curl 获取示例数据 <https://microsoftedge.github.io/Demos/json-dummy-data/64KB.json>, 再用 jq 提取 version 大于 6 的人员姓名。(提示: 先 jq . 看结构; 再试 jq '.[] | select(...) | .name')

解析：这一题说明结构化数据最好用结构化工具处理。先通过 jq . 熟悉 JSON 结构，再筛选满足条件的记录并取出姓名字段，是 JSON 数据分析的标准流程。

```
curl -s 'https://microsoftedge.github.io/Demos/json-dummy-data/64KB.json' |  
jq '.[] | select(.version|tonumber > 6) | .name'
```



```
luoyue@LAPTOP-20PGDGAI:~$ curl -s 'https://microsoftedge.github.io/Demos/json-dummy-data/64  
KB.json' | jq '.[] | select(.version|tonumber > 6) | .name'  
Command 'jq' not found, but can be installed with:  
sudo snap install jq # version 1.5+dfsg-1, or  
sudo apt install jq # version 1.7.1-3ubuntu0.24.04.2  
See 'snap info jq' for additional versions.
```

图 22: 练习 16: jq 过滤 JSON 数据

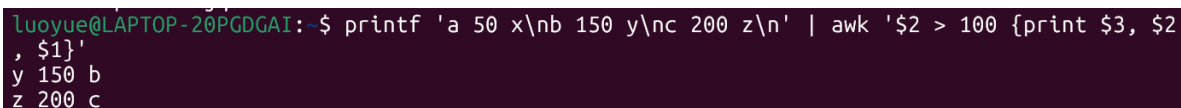
6.15 练习 17

题目：awk 可以按列值过滤行并改写输出。例如,awk '\$3 /pattern/ { \$4="" ; print }' 会只输出第三列匹配 pattern 的行，并省略第四列。请写一个 awk 命令：只输出第二列大于 100 的行，并交换第一列和第三列。可用这条命令测试：

```
printf 'a 50 x\nb 150 y\nc 200 z\n' | awk '$2 > 100 {print $3, $2, $1}'
```

解析：awk 非常适合列式文本处理。该题要求按第二列过滤，再重排输出顺序，本质上就是在条件筛选基础上做字段重组。

```
printf 'a 50 x\nb 150 y\nc 200 z\n' | awk '$2 > 100 {print $3, $2, $1}'
```



```
luoyue@LAPTOP-20PGDGAI:~$ printf 'a 50 x\nb 150 y\nc 200 z\n' | awk '$2 > 100 {print $3, $2  
, $1}'  
y 150 b  
z 200 c
```

图 23: 练习 17: awk 条件过滤与列重排

6.16 练习 18

题目：拆解讲义中的 <https://missing-semester-cn.github.io/2026/course-shell/#shell-%E8%AF%AD%E8%A8%80bash> SSH 日志处理管道：每一步分别做了什么？然后仿照它构建一个管道，从 `~/.bash_history`（或 `~/.zsh_history`）中找出你最常使用的 Shell 命令。

解析：这一题综合了管道分析、文本清洗、排序和频次统计。通过拆解讲义中的 SSH 日志处理思路，可以理解每一步如何把原始日志逐步压缩成统计结果；再把同样方法迁移到历史命令文件，就能得到自己最常使用的命令分布。

```
awk '{print $1}' ~/.bash_history | sort | uniq -c | sort -nr | head
```

```

luoyue@LAPTOP-20PGDGAI:~$ awk '{print $1}' ~/.bash_history | sort | uniq -c | sort -nr | head
88 git
73 cd
47 .venv-wsl/bin/python
45 docker
29 echo
29 cat
26 ls
24 sudo
23
22 python3

```

图 24: 练习 18: Shell 历史命令统计

7 Git 提交记录

本次实验代码、截图和报告均通过 Git 进行管理，并上传到个人仓库：

<https://github.com/Llj159-lab/sys-dev-tools-lab>

实验过程中应按题目和阶段分多次提交，避免单次全量提交。这样既便于回溯修改过程，也能更清楚地体现每一题的完成状态。

```

luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week01/q04$ cd ~/sys-dev-tools-lab
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab$ git remote -v
origin https://github.com/Llj159-lab/sys-dev-tools-lab.git (fetch)
origin https://github.com/Llj159-lab/sys-dev-tools-lab.git (push)
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab$ git status --short --branch
## main...origin/main
M week01/q04/report.fdb_latexmk
M week01/q04/report.flx
M week01/q04/report.log
M week01/q04/report.pdf
?? week01/q04/report.xdv
?? week01/report_assets/
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab$ git log --oneline --decorate --graph --all --date-order
* 5344263 (HEAD -> main, origin/main) fix(week01): correct q01 and q03 results
* 6d089b8 feat(week01): 完成q04 LaTeX报告
* e37eddb feat(week01): 完成q03 Git冲突实验
* ffa94e0 feat(week01): 完成q01文件整理
* 81dccb3 feat(week01): 完成q02日志统计脚本

```

图 25: GitHub 远程仓库与 commit 记录

8 实验总结与感悟

本次第一周实验围绕 Shell、Git 和 LaTeX 三条主线展开，四道课上实验分别对应文件系统整理、文本统计、分支冲突和文档排版，彼此之间形成了完整的工具链训练。通过这些题目可以看到，命令行工具的价值不仅在于单个命令是否熟悉，更在于能否把多个小工具按正确顺序组合起来，形成可复现、可验证的工作流程。

Shell 部分的训练帮助我建立了对路径、权限、重定向、条件判断和管道的整体认识。文件名中包含空格、隐藏文件需要特别处理，标准流需要区分 stdout 和 stderr，脚本需要

考虑参数、退出状态与执行权限，这些都是实际使用中最容易出错的地方。通过这些练习，我更加明确了“细节决定可用性”这一点。

Git 和 LaTeX 的练习则分别从版本管理和文档表达两个角度强化了工程化意识。Git 冲突说明分支历史必须清晰，合并结果必须能被准确追溯；LaTeX 则说明结构化写作可以减少手工编号和版面错误，提高报告的规范程度。整体来看，这一周的内容让我对开发工具的理解从“会用命令”推进到了“理解工具如何协作完成任务”。